Memoria de diseño

Cómo y por qué llegamos a este modelo entidad-relación — Consultorio Santa María Josefa

Este documento resume el camino que seguimos durante la Etapa 2 (Diseño y Planificación) para llegar al modelo entidad-relación final. No es solo el resultado, sino las razones detrás de cada decisión: qué problema resolvía, qué le faltaba a la versión anterior y por qué la cambiamos. Lo escribimos como estudiantes de primer año, en base a lo que hemos visto hasta ahora en el Taller de Base de Datos.

1. Punto de partida: el diagnóstico

En la Etapa 1 (Diagnóstico) definimos el problema del socio comunitario: el consultorio "Santa María Josefa" no tenía ningún sistema para controlar sus insumos, y eso generaba dos riesgos concretos: quedarse sin insumos críticos durante una consulta, y que insumos se vencieran sin que nadie se diera cuenta. De ahí salieron los tres objetivos de la actividad: (1) registrar el 100% de los insumos usados por box, especialidad y profesional, (2) generar reportes que reduzcan el desfase entre el stock real y el registrado, y (3) alertar automáticamente cuando el stock esté bajo el mínimo o un insumo esté por vencer. Todo el diseño que sigue apunta a cumplir esos tres objetivos.

2. Primera versión del modelo

Nuestra primera versión tenía ocho entidades: `Especialidad`, `Personal` (luego renombrada `Staff`), `Box`, `Proveedor`, `Categoria_Insumo`, `Insumo`, `Servicio` (que al principio llamamos `Consumo`) y `Alerta`. Con esto cubríamos lo básico del objetivo 1: qué insumo se usó, en qué box, en qué consulta y con qué profesional. Pero al revisarlo con más calma, encontramos varios vacíos frente a los objetivos 2 y 3.

3. Por qué agregamos el ingreso de insumos

Nos dimos cuenta de que el modelo solo registraba cómo *salían* los insumos (en cada consulta), pero no cómo *entraban*. Sin eso, el objetivo 2 —reducir el desfase entre el stock real y el registrado— no se podía cumplir: no hay forma de calcular un stock si solo se cuenta lo que se consume y nunca lo que se recibe. Por eso agregamos `Ingreso_insumo`, con el proveedor, el personal que lo registra, si es compra o donación, y la fecha.

4. Por qué separamos las tablas de detalle

Al principio pensamos en guardar los insumos usados directamente como columnas dentro de `Servicio` e `Ingreso_insumo`. El problema es que una sola consulta casi siempre usa más de un insumo (gasas, guantes, alcohol gel, etc.), y una tabla no puede tener "una lista de insumos" en un solo campo sin romper la primera forma normal (1FN): cada campo debe guardar un solo valor. La solución fue crear una fila por cada insumo usado, en una tabla aparte: `Detalle_Servicio` y, para lo mismo pero con los ingresos, `Detalle_Ingreso`. De paso, esto resuelve algo que en clases vimos como relación

"muchos a muchos": una consulta usa varios insumos y un insumo se usa en varias consultas, y una tabla intermedia con las dos llaves es la forma correcta de modelar eso.

5. Por qué agregamos la salida de insumos

Al revisar de nuevo el objetivo 2, notamos otro vacío: si un insumo se vence, se rompe, o simplemente un conteo físico muestra menos de lo que el sistema registra, no había ninguna tabla donde anotar esa salida. Sin eso, el stock registrado se iba a seguir desfasando del stock real, que es justo lo que el objetivo 2 pide evitar. Por eso agregamos `Salida_insumo` y su tabla de detalle `Detalle_Salida`, con un motivo (vencimiento, merma o ajuste de conteo). Con esto, `Insumo.stock_actual` queda definido de una forma simple y completa: lo que entró por ingresos, menos lo que se usó en consultas, menos lo que salió por estas otras razones.

6. Por qué agregamos la notificación y el canal de comunicación

El objetivo 3 no solo pide detectar el problema (stock bajo o vencimiento próximo), sino además notificar al personal y al proveedor. Con solo la tabla `Alerta` podíamos guardar que el problema existía, pero no a quién se avisó, por qué medio, ni si el aviso realmente se envió. Por eso agregamos `Canal_Comunicacion` (correo, WhatsApp, etc.) y `Notificacion`, que conecta una alerta con un canal y con un destinatario, que puede ser un miembro del staff o un proveedor.

7. La normalización y el error que corregimos

Cuando aplicamos el proceso de normalización (1FN a 3FN) a todas las tablas, encontramos un solo problema real: `Notificacion` tenía un campo `tipo_destinatario` que decía si el destinatario era "staff" o "proveedor". Pero ese dato no es realmente información nueva: se puede saber mirando cuál de las dos llaves, `id_staff` o `id_proveedor`, tiene un valor. Guardar ese campo aparte era una dependencia transitiva, que es justo lo que la tercera forma normal (3FN) no permite. Lo eliminamos, y en su lugar dejamos una regla en la base de datos (una restricción `CHECK`) que obliga a que siempre haya exactamente un destinatario, ni los dos ni ninguno.

8. Por qué la alerta tiene estados

Nos dimos cuenta de que una alerta no es algo que pasa una sola vez: se genera, después se notifica, después alguien la resuelve (o la descarta si fue un error). Si el campo `estado` solo existiera sin valores definidos, cada uno de nosotros podría usarlo de forma distinta al programar las consultas SQL. Por eso definimos un ciclo de vida fijo: **pendiente → notificada → resuelta**, con la opción de pasar a **descartada** desde pendiente o desde notificada.

9. Cómo verificamos que el modelo cumple los objetivos

Antes de dar por cerrado el diseño, revisamos el modelo objetivo por objetivo:

- **Objetivo 1** (registrar insumos por box, especialidad y profesional): se cumple con `Servicio`, `Detalle_Servicio`, `Box` y `Staff`. La especialidad se obtiene por el box o por el staff de la consulta.
- **Objetivo 2** (reportes y menos desfase de stock): se cumple con consultas SQL sobre `Detalle_Servicio`, y el desfase se reduce porque ahora `Ingreso_insumo` y `Salida_insumo` registran todos los movimientos de stock, no solo el consumo.
- **Objetivo 3** (alerta automática a personal y proveedor): se cumple con `Alerta`, `Notificacion` y `Canal_Comunicacion`, usando `stock_minimo` y `fecha_vencimiento` de `Insumo` para detectar el problema.

10. De la teoría a la base real

Por último, pasamos el modelo a un script de PostgreSQL con las quince tablas, sus llaves primarias y foráneas, y las restricciones que definimos durante el diseño (cantidades positivas, estados válidos, la regla del destinatario único). Cargamos datos de prueba pensados a propósito para que algunas alertas se generen (insumos bajo el mínimo, insumos por vencer) y comprobamos que el stock se calculaba bien después de un ingreso, una consulta y una salida. Que la base de datos funcionara con datos reales fue, para nosotros, la confirmación de que el diseño estaba bien pensado y no solo en el papel.

Conclusión

El modelo final tiene quince tablas, todas conectadas por relaciones de uno a muchos, y llegamos a él no de una vez, sino revisando el diseño contra los tres objetivos varias veces y agregando lo que faltaba cada vez que encontramos un caso que el modelo anterior no cubría. Nos sirvió sobre todo entender que un modelo entidad-relación no se construye una sola vez y ya, sino que se va corrigiendo a medida que se revisa con más cuidado lo que el socio comunitario realmente necesita.